

CS 225

Data Structures

Volodymyr Kindratenko

Slides courtesy of Wade Fagen-Ulmschneider

Hashing

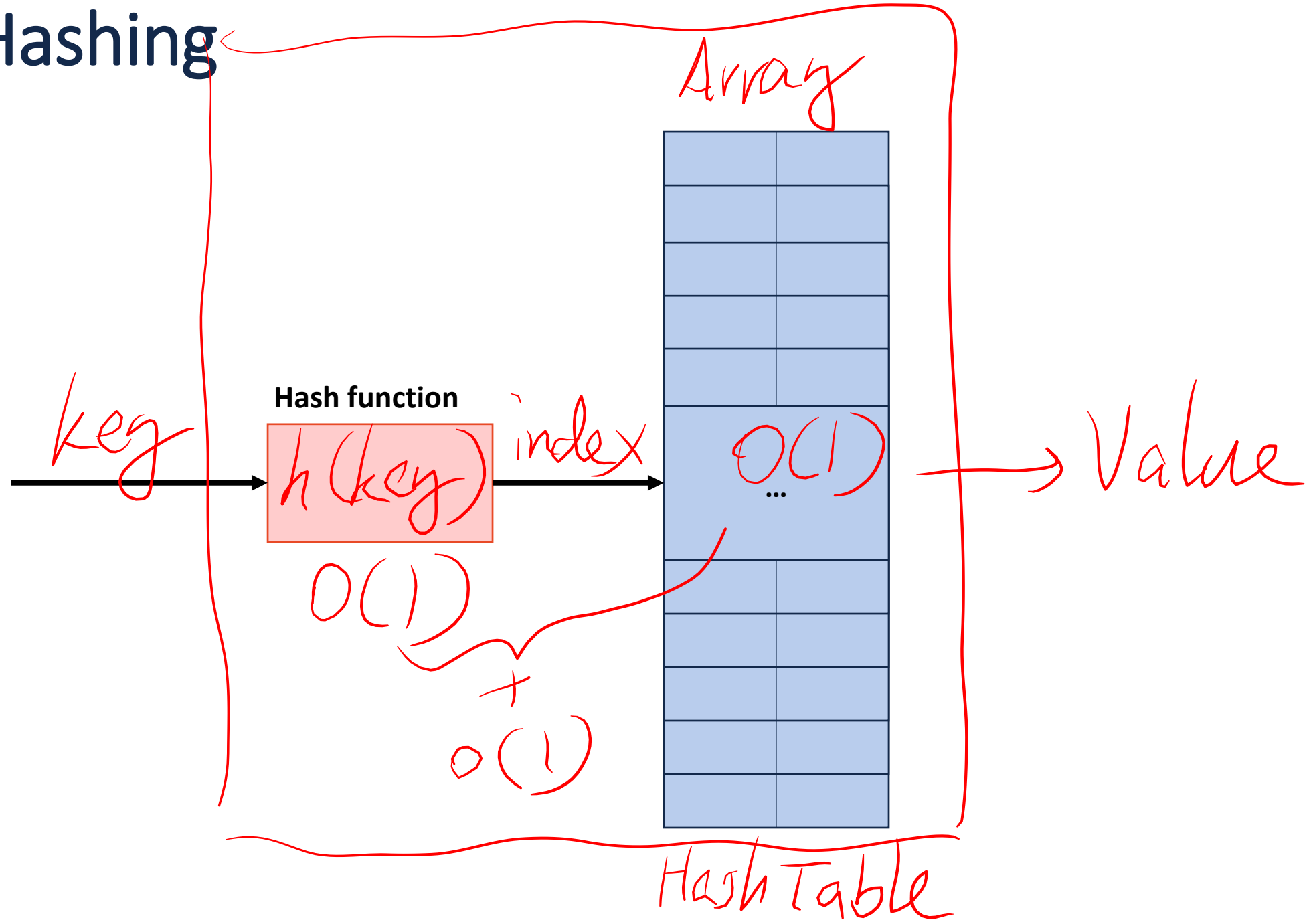
Goals:

We want to define a **keyspace**, a (mathematical) description of the keys for a set of data.

K
...use a function to map the **keyspace** into a small set of integers.

$$h(\text{key}) : \text{key} \rightarrow \mathbb{N}$$

Hashing



A Hash Table based Dictionary

Client Code:

```
1 Dictionary<KeyType, ValueType> d;  
2 d[k] = v;
```

A **Hash Table** consists of three things:

1. A hash function ★
2. An array
3. ?? mystery }

A Perfect Hash Function

bijective function

(Angrave, CS 241)

(Beckman, CS 421)

(Cunningham, CS 210)

(Davis, CS 101)

(Evans, CS 126)

(Fagen-Ulmschneider, CS 225)

(Gunter, CS 422)

~~(Herman, CS 233)~~

Harris, CS 105

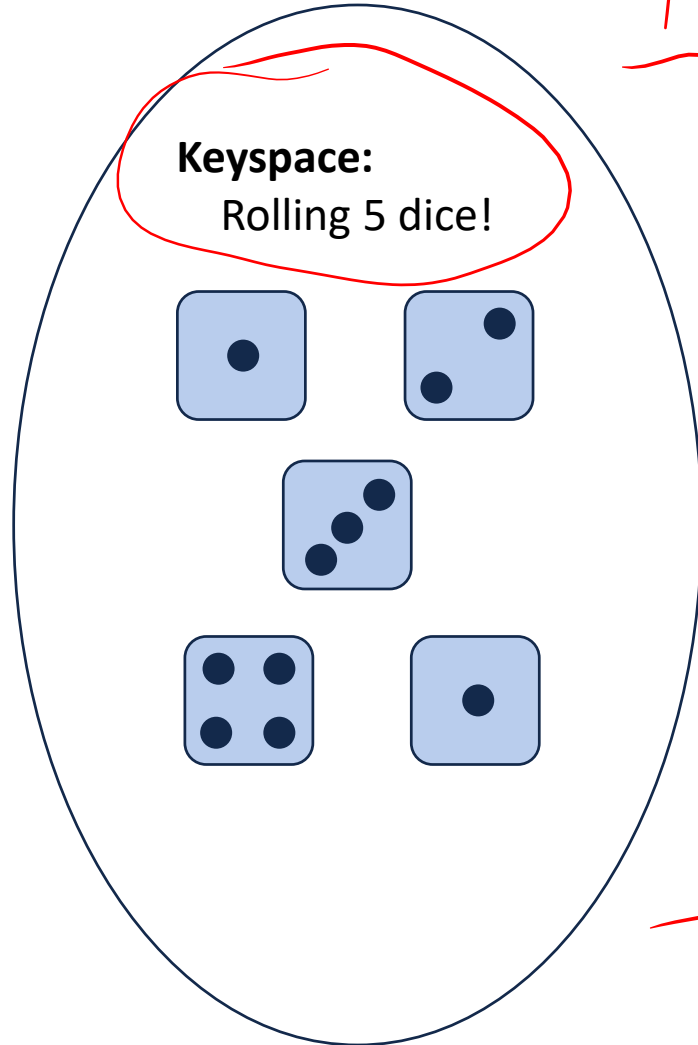
Hash function

key[0] - 'A'

Key	Value
0	Angrave
1	Beckman
2	...
3	...
4	...
5	...
6	...
7	...

onto

A Perfect Hash Function



1-to-1
No

Hash function



Petals around the rose

Onto
No: $h^{-1}(1)$
has no value

$$P(20) = \frac{1}{6^5}$$

$$P(0) > \frac{1}{6^5}$$

Key	Value
0	✓
1	X
2	✓
3	X
4	
5	X
6	
7	X
8	
9	X
10	
11	X
12	
13	X
14	
15	X

Hash Function

Our **hash function** consists of two parts:

- A **hash**: $h_h \text{ key} \rightarrow i \text{ (int)}$

- A **compression**:

$$h_c: i \rightarrow [0, N-1] \boxed{\% N}$$

Choosing a good hash function is tricky...

- Don't create your own (yet*)
- Very smart people have created very bad hash functions

Hash Function

Characteristics of a good hash function:

1. Computation Time:

Runs in constant time, $O(1)$

2. Deterministic:

No random element
if $k_1 == k_2 \rightarrow h(k_1) == h(k_2)$

3. Satisfy the SUHA:

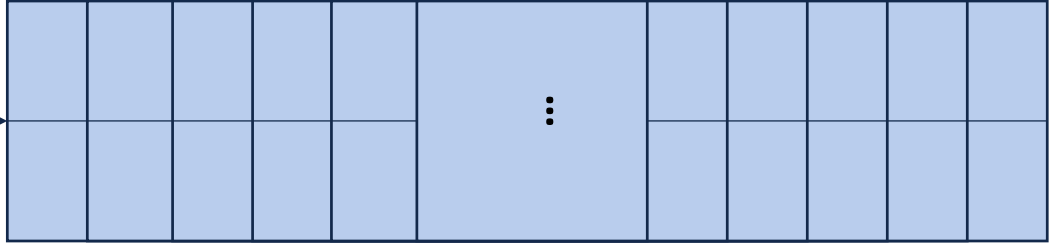
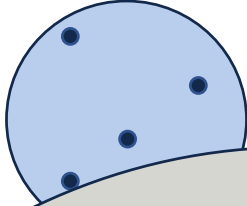
Simple Uniform hashing assumption

For keys k, j
where $k \neq j$

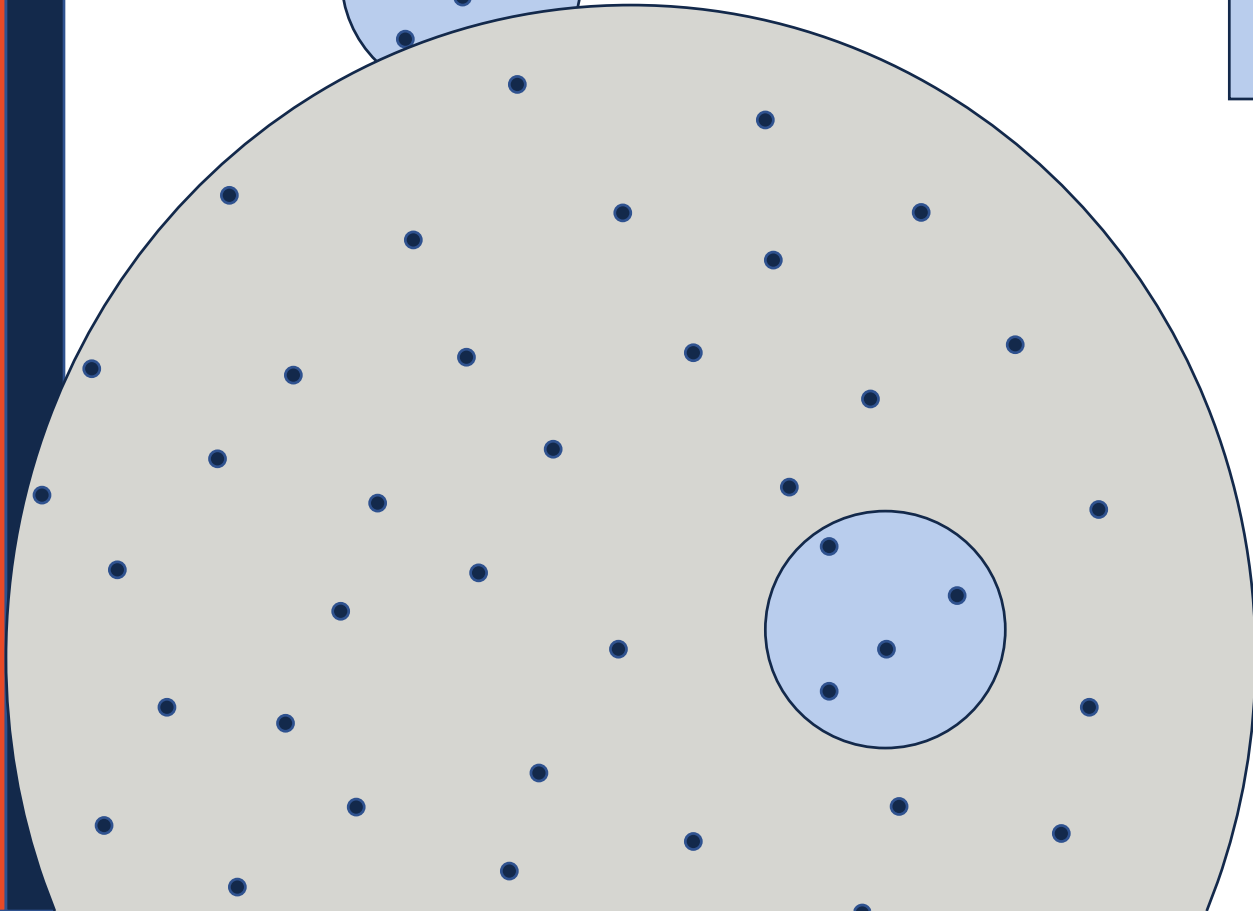
$$P(h(k) == h(j)) = 1/N$$

General Purpose Hash Function

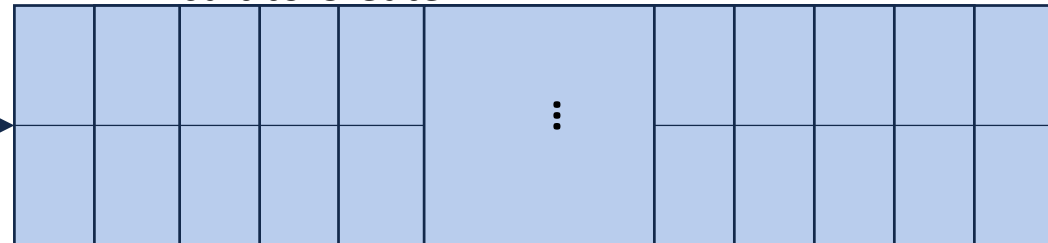
Keyspaces



Easy to create if: $|\text{KeySpace}| \sim N$



Difficult to Create:



Hash Function

Given: Easy to create a hash function of strings of length 8

Hash Function

Idea: Map 40 character things to length 8:

Alice was beginning to get very tired of sitting by her sister on the bank, and of having nothing to do: once or twice she had peeped into the book her sister was reading, but it had no pictures or conversations in it, 'and what is the use of a book,' thought Alice 'without pictures or conversations?' So she was considering in her own mind (as well as she could, for the hot day made her feel very sleepy and stupid), whether the pleasure of making a daisy-chain would be worth the trouble of getting up and picking the daisies, when suddenly a White Rabbit with pink eyes ran close by her. There was nothing so very remarkable in that; nor did Alice think it so very much out of the way to hear the Rabbit say to itself, 'Oh dear! Oh dear! I shall be late!' (when she thought it over afterwards, it occurred to her that she ought to ha

→ i w e i t e g r

→ _ _ p t h o e s

Hash Function

Idea: Map 40 character things to length 8:

`https://en.wikipedia.org/wiki/Main_Page`

`https://en.wikipedia.org/wiki/Battle_of`

`https://en.wikipedia.org/wiki/Vector_Gen`

`https://en.wikipedia.org/wiki/2017_Austr`

`https://en.wikipedia.org/wiki/19th_Natio`

`https://en.wikipedia.org/wiki/Japanese_g`

→ s n k d g / - e

→ s n k d g / - i

Hash Function

In CS 225, we focus on **general purpose** hash functions.

Other hash functions exist with different properties
(eg: cryptographic hash functions)



CS 225

Data Structures

Volodymyr Kindratenko

Slides courtesy of Wade Fagen-Ulmschneider

A Hash Table based Dictionary

Client Code:

```
1 Dictionary<KeyType, ValueType> d;  
2 d[k] = v;
```

A **Hash Table** consists of three things:

1. hash function $h(k)$
2. array
3. collision resolution strategies

Collision Handling: Separate Chaining

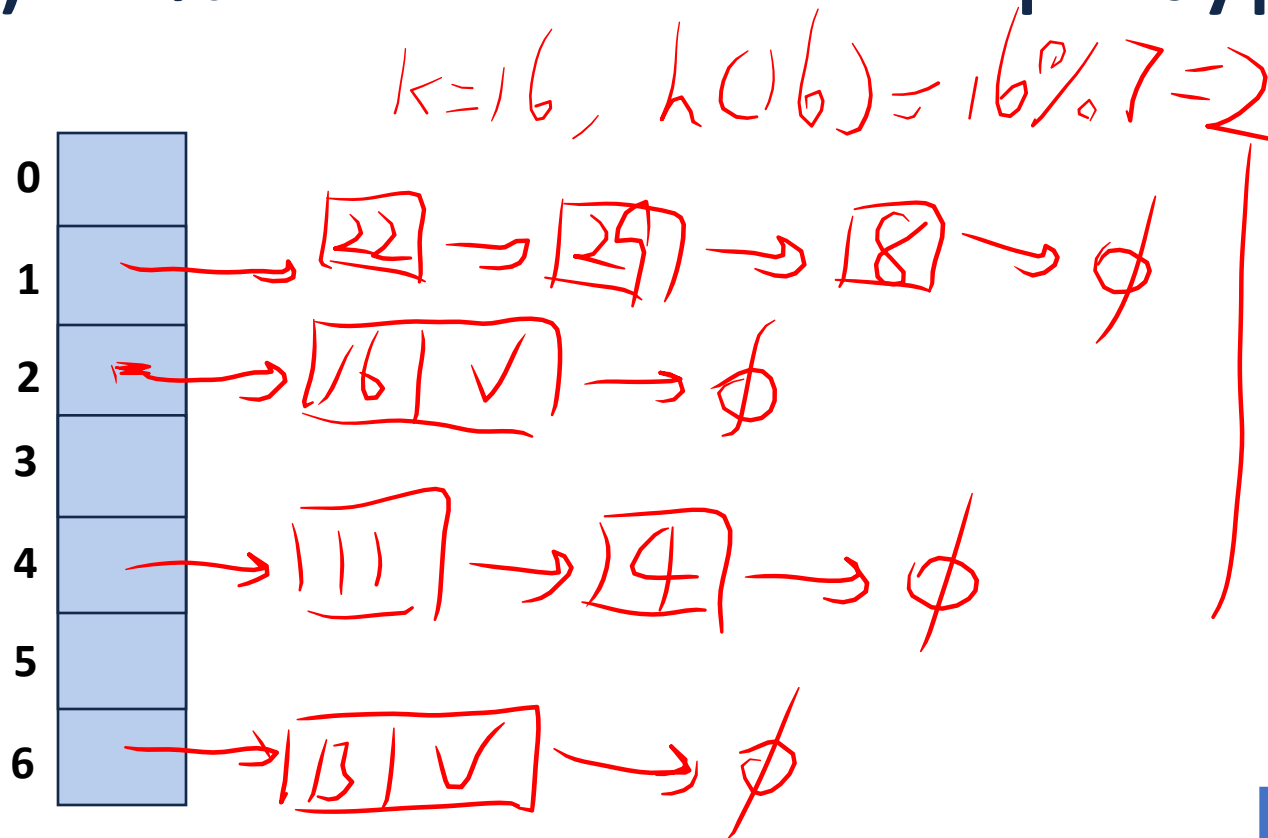
$S = \{ 16, 8, 4, 13, 29, 11, 22 \}$

$|S| = n$

Linked list off every node

$h(k) = k \% 7$

$|Array| = N$



load factor

$$\alpha = n/N$$

$\alpha \Rightarrow$ unbounded

open hashing

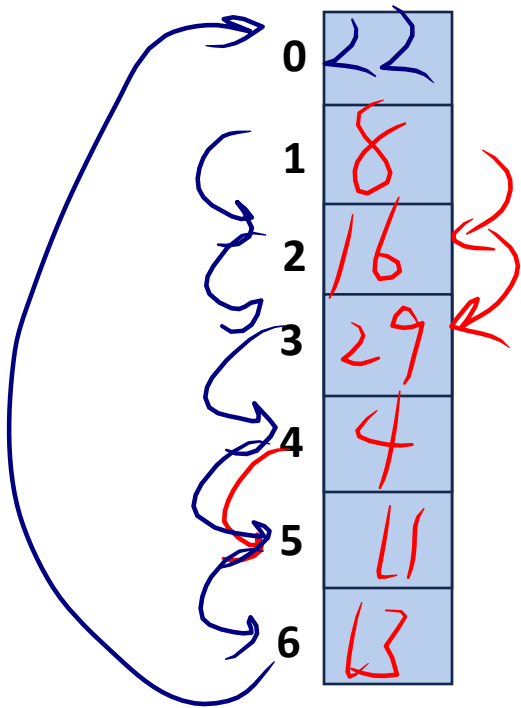
	Worst Case	SUHA
Insert	$O(1)$	
Remove/Find	$O(n) \equiv O(n/N)$	

Collision Handling: Probe-based Hashing

$S = \{ 16, 8, 4, 13, 29, 11, 22 \}$ $|S| = n$

$h(k) = k \% 7$

$|Array| = N$



Try $h(k) = (k + 0) \% 7$, if full...

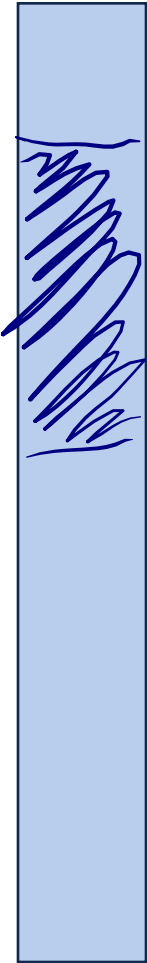
Try $h(k) = (k + 1) \% 7$, if full...

Try $h(k) = (k + 2) \% 7$, if full...

Try ... $2 \leq i < \infty$ $\frac{n}{N} = \frac{7}{7}$

A Problem w/ Linear Probing

Primary clustering:



Description:

hash locations tend to cluster in primary clusters even under low load α

Remedy:

double hashing

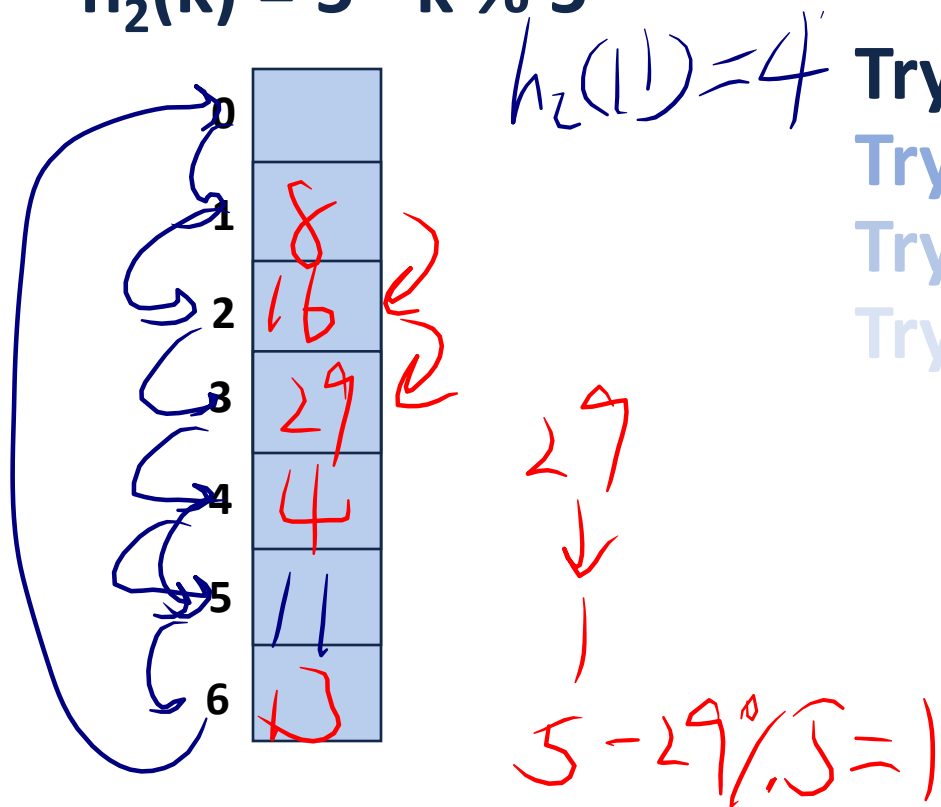
Collision Handling: Double hashing

$S = \{ 16, 8, 4, 13, 29, 11, 22 \}$ $|S| = n$

$$h_1(k) = k \% 7$$

$$|\text{Array}| = N$$

$$h_2(k) = 5 - k \% 5$$



Try $h(k) = (k + 0 * h_2(k)) \% 7$, if full...

Try $h(k) = (k + 1 * h_2(k)) \% 7$, if full...

Try $h(k) = (k + 2 * h_2(k)) \% 7$, if full...

Try ...

$$h(k, i) = (h_1(k) + i * h_2(k)) \% 7$$

Running Times

The expected number of probes for find(key) under SUHA

Linear Probing:

- Successful: $\frac{1}{2}(1 + 1/(1-\alpha))$
- Unsuccessful: $\frac{1}{2}(1 + 1/(1-\alpha))^2$

Double Hashing:

- Successful: $1/\alpha * \ln(1/(1-\alpha))$
- Unsuccessful: $1/(1-\alpha)$

Separate Chaining:

- Successful: $1 + \alpha/2$
- Unsuccessful: $1 + \alpha$

(Don't memorize these equations, no need.)

Instead, observe:

- As α increases:

We have slower hash table operations

- If α is constant:

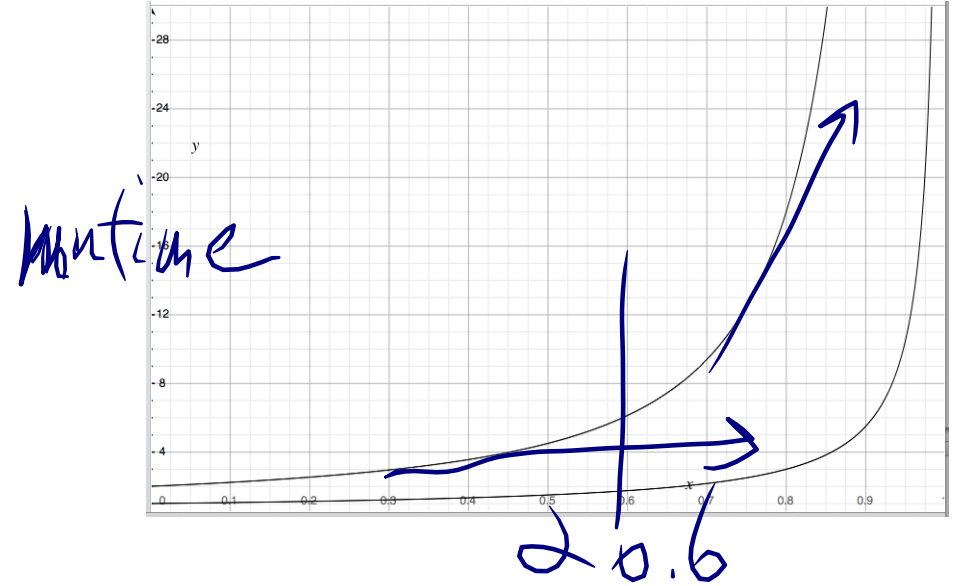
Running time does not change

Running Times

The expected number of probes for find(key) under SUHA

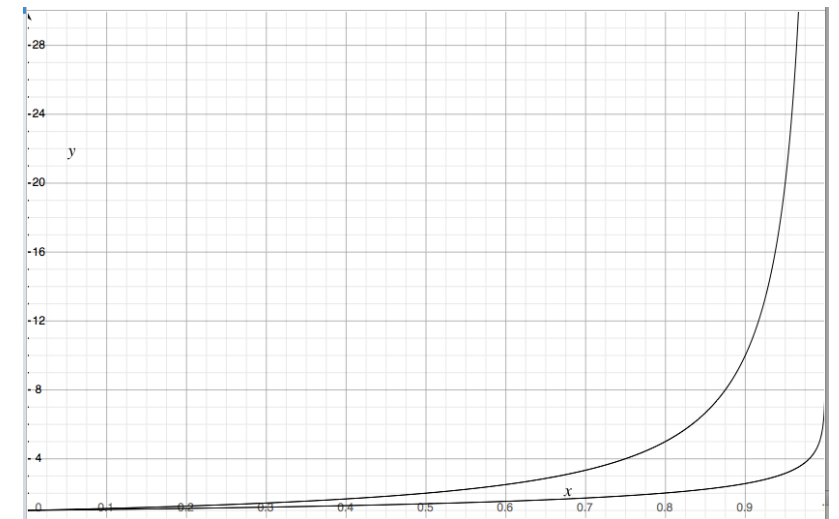
Linear Probing:

- Successful: $\frac{1}{2}(1 + \frac{1}{1-\alpha})$
- Unsuccessful: $\frac{1}{2}(1 + \frac{1}{1-\alpha})^2$



Double Hashing:

- Successful: $\frac{1}{\alpha} * \ln(\frac{1}{1-\alpha})$
- Unsuccessful: $\frac{1}{1-\alpha}$





CS 225

Data Structures

Volodymyr Kindratenko

Slides courtesy of Wade Fagen-Ulmschneider

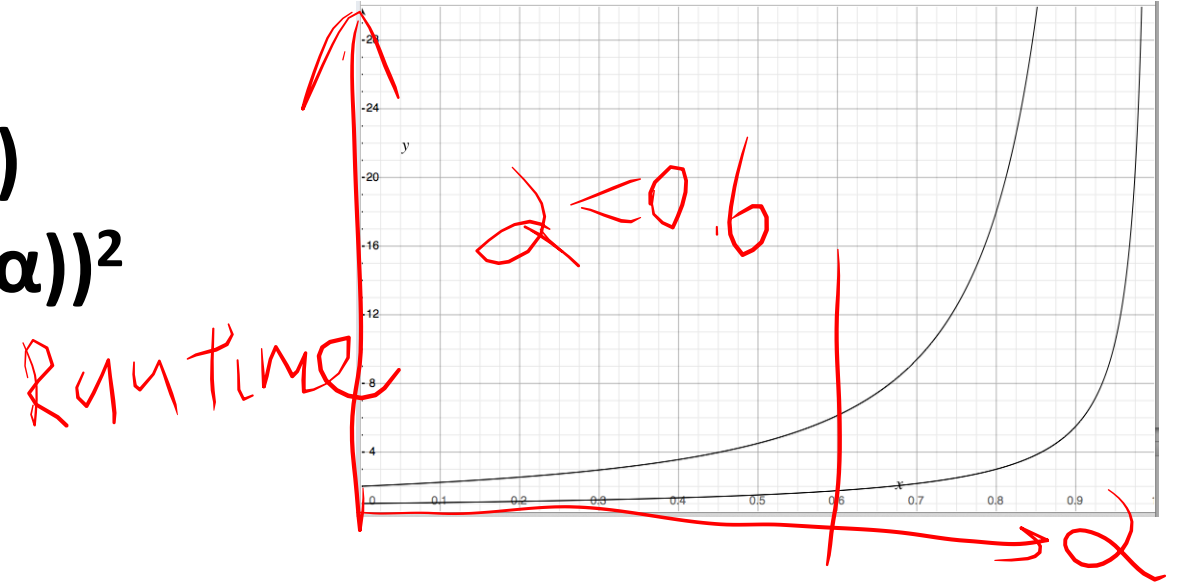
Running Times

The expected number of probes for find(key) under SUHA

$n = 100$
 $n = 100000000$

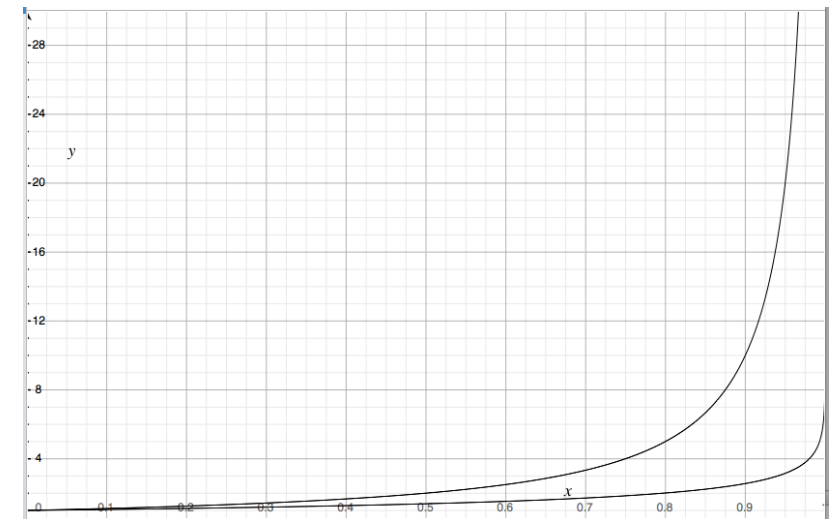
Linear Probing:

- Successful: $\frac{1}{2}(1 + 1/(1-\alpha))$
- Unsuccessful: $\frac{1}{2}(1 + 1/(1-\alpha))^2$



Double Hashing:

- Successful: $1/\alpha * \ln(1/(1-\alpha))$
- Unsuccessful: $1/(1-\alpha)$



ReHashing

~~What if the array fills?~~

What should happen
as $2 \rightarrow 0.6$?

4
8
14
11
22
6
13

$$h(k) = k \% 7$$

0	11
1	22
2	13
3	14
4	4
5	
6	6
7	
8	8
9	
10	

Double size
update hash f

Copy elements

- Rehash elements

- Make to new location $0(1)$

$$k \% 11 \quad \downarrow \quad 0(1)$$

Running Times

	Hash Table	AVL	Linked List
Find	Amortized: $O(1)$ Worst Case: $O(n)$	$O(\lg(n))$	$O(n)$
Insert	Amortized: $O(1)$ Worst Case: $O(n)$	$O(\lg(n))$	$O(1)$
Storage Space	$N = \frac{n}{0.6} \approx 1.5n$ $O(n)$	$O(n)$ $2 \times \text{ptrs/elem}$	$O(n)$ $1 \times \text{ptr/elem}$

Which collision resolution strategy is better?

- Big Records:

SC $\rightarrow$ "Open Hashing"

- Structure Speed:

Double Hashing $\rightarrow$ "closed Hashing"

What structure do hash tables replace?

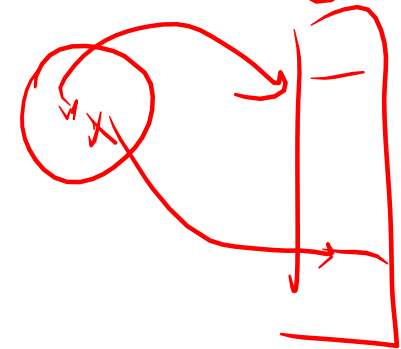
List, AVL

What constraint exists on hashing that doesn't exist with BSTs?

We need the key, AVL can find neighboring keys

Why talk about BSTs at all?

* Nearest neighbor
Ranges [10, 100]



std data structures

std::map

operator[]

insert

erase

std data structures

std::map

::operator[]

::insert

::erase

::lower_bound(key) ➔ Iterator to first element $\leq$ key

::upper_bound(key) ➔ Iterator to first element $>$ key

→ Impl w/ a balanced BST

std data structures

std::unordered_map

::operator[]

::insert

::erase

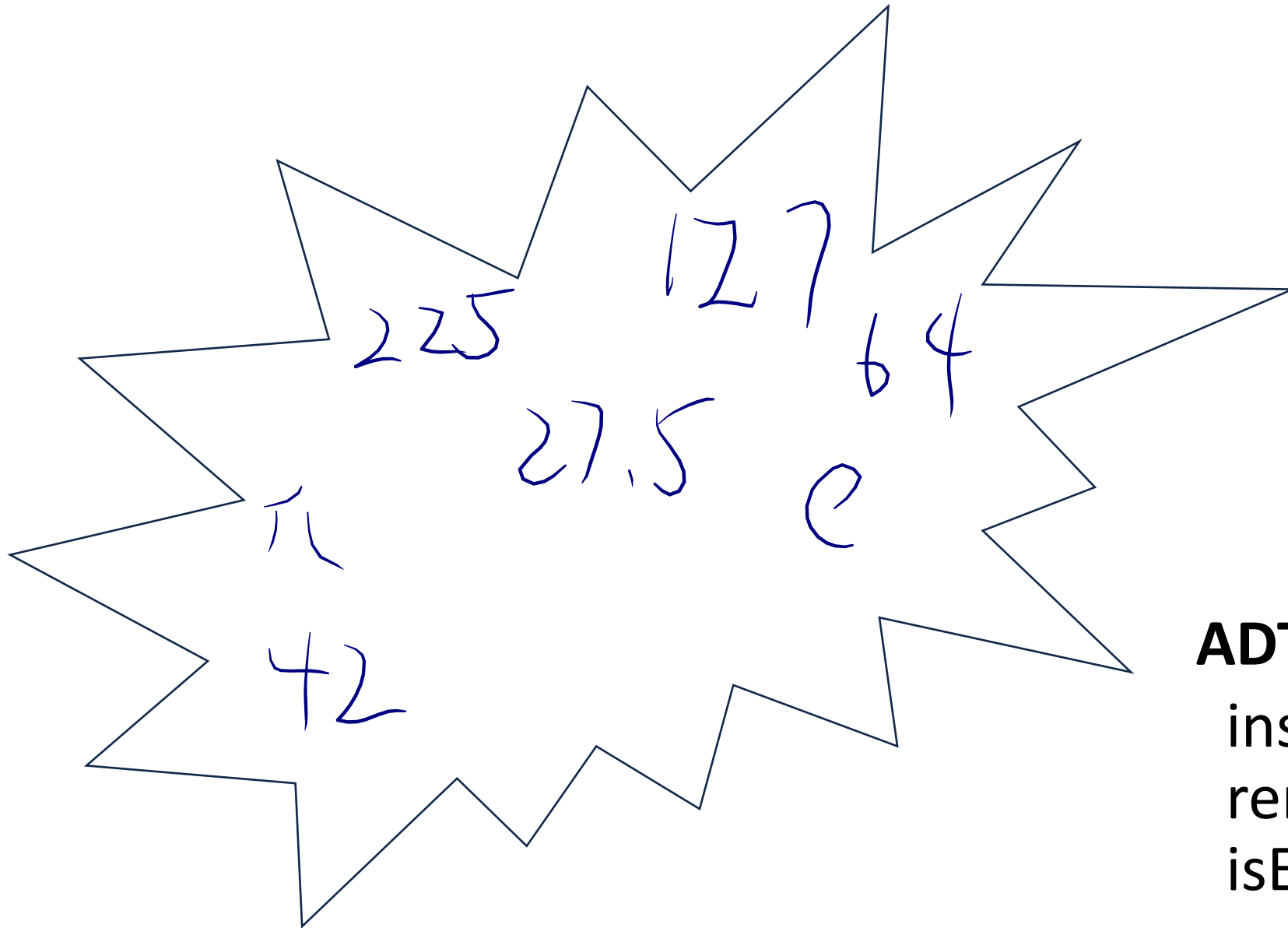
~~— ::lower_bound(key) → Iterator to first element $\leq$ key~~

~~— ::upper_bound(key) → Iterator to first element $>$ key~~

::load_factor()

::max_load_factor(ml) ➔ Sets the max load factor

Secret, Mystery Data Structure



ADT:

insert

remove

isEmpty

min()

Priority Queue Implementation

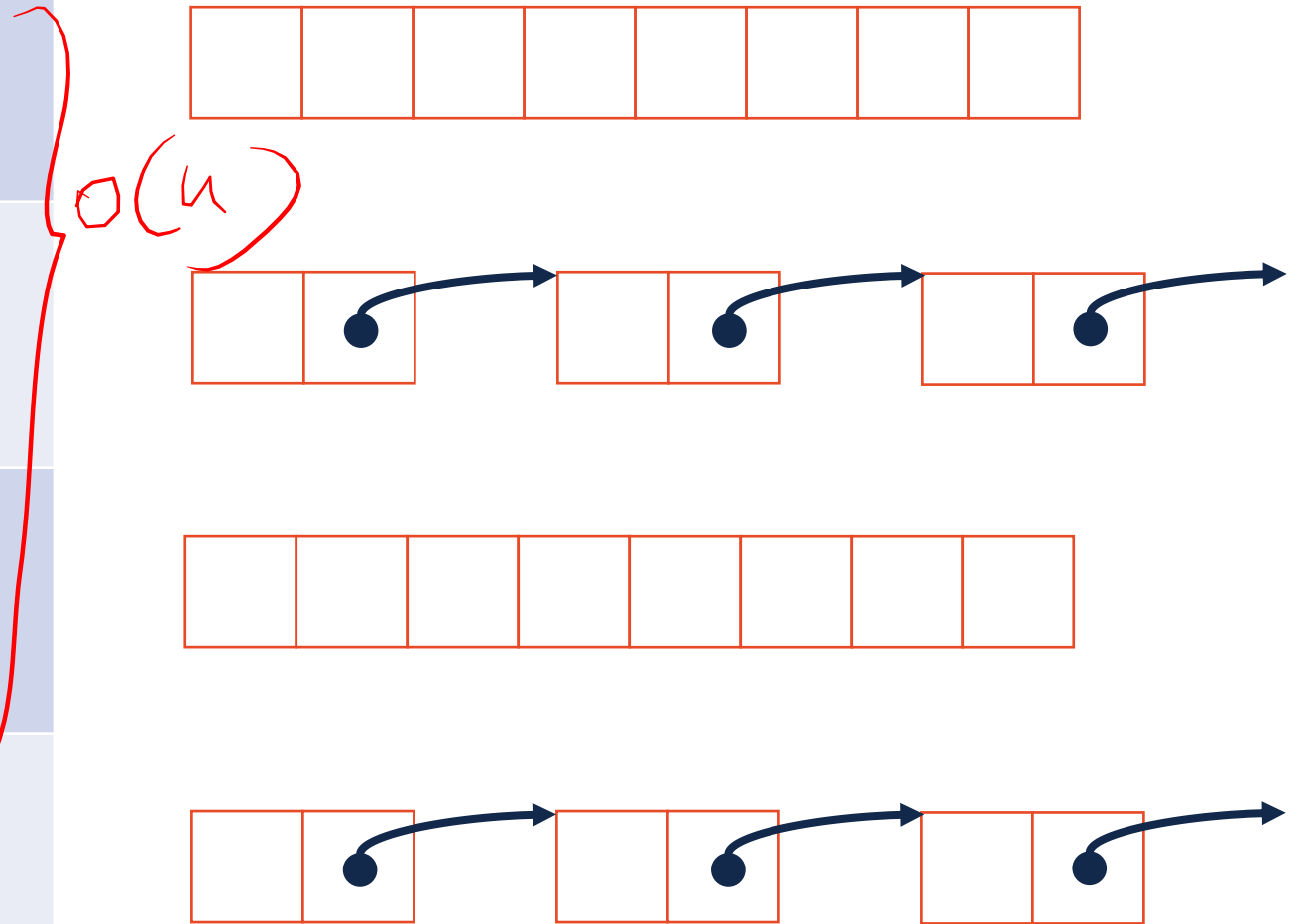
insert	removeMin
$O(n)$ $O(1)$	$O(n)$
$O(1)$	$O(n)$
$O(\lg(n))$ $O(1)$	$O(1)$
$O(\lg(n))$ $O(1)$	$O(1)$

find $O(\lg n)$
 insert push $O(1)$
 find $O(1)$
 insert $O(1)$
 $O(n)$



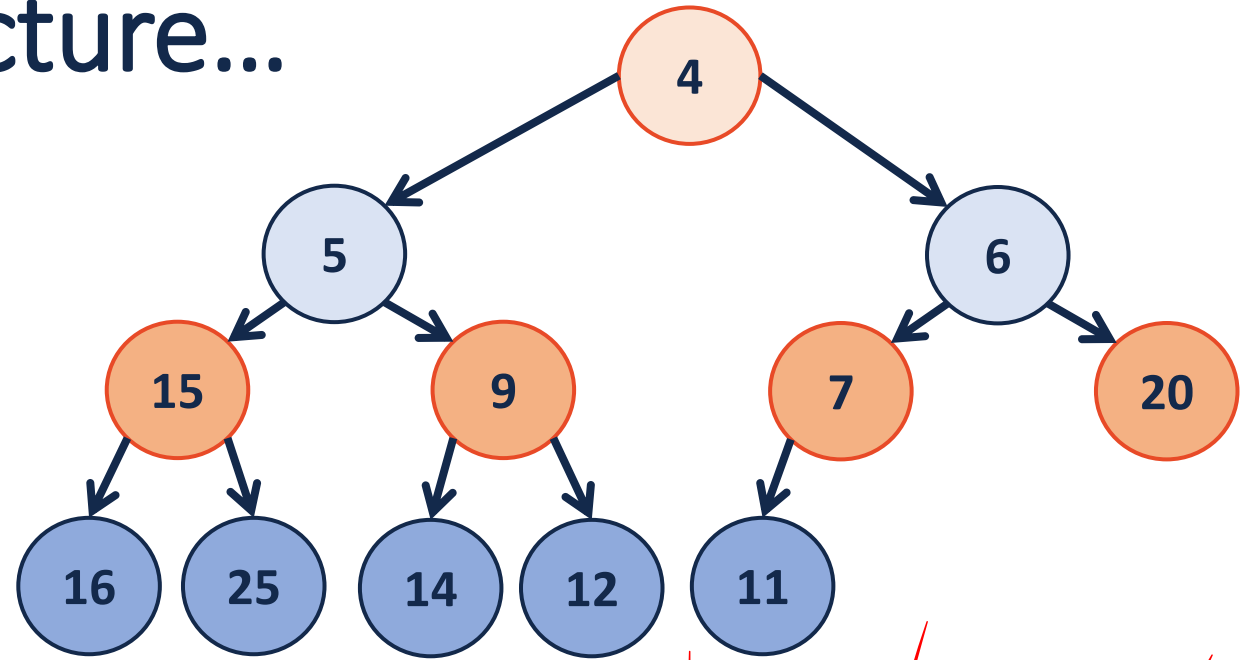
Priority Queue Implementation

insert	removeMin
$O(1)$	$O(n)$
$O(1)$	$O(n)$
$O(n)$	$O(1)$
$O(n)$	$O(1)$



Another possible structure...

Binary Tree
- Complete
- ! BST

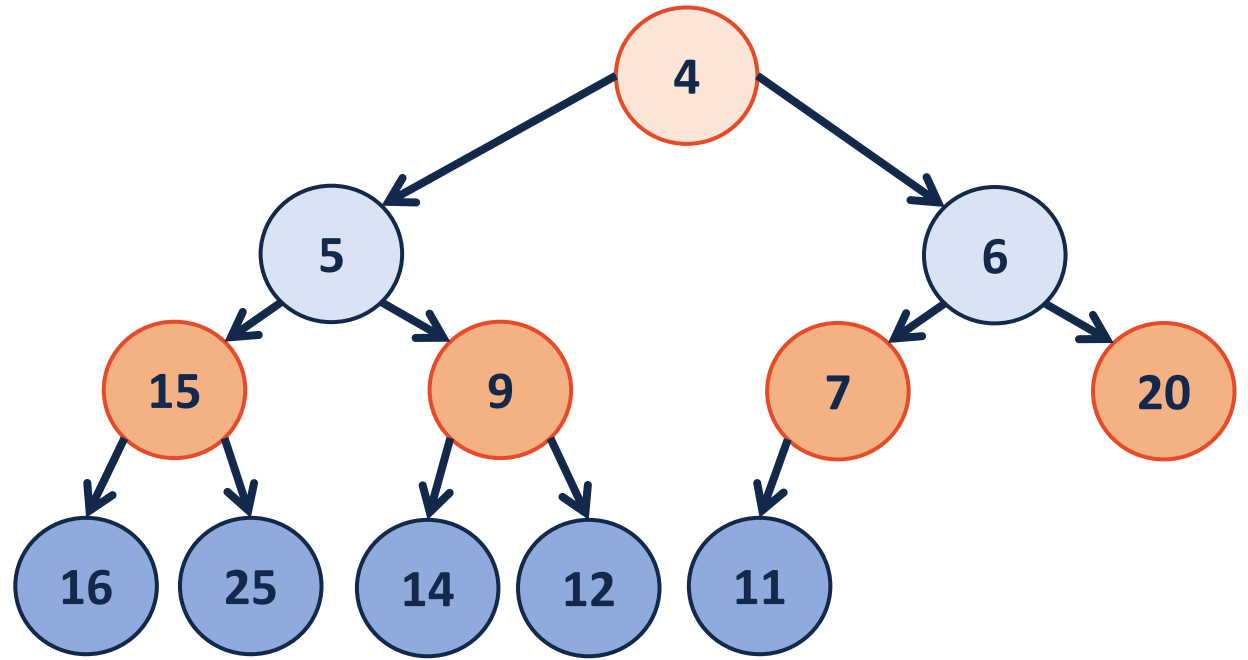


- All descendants of a node > the value of the node
↳ Root is the smallest element

(min)Heap

A complete binary tree T is a min-heap if:

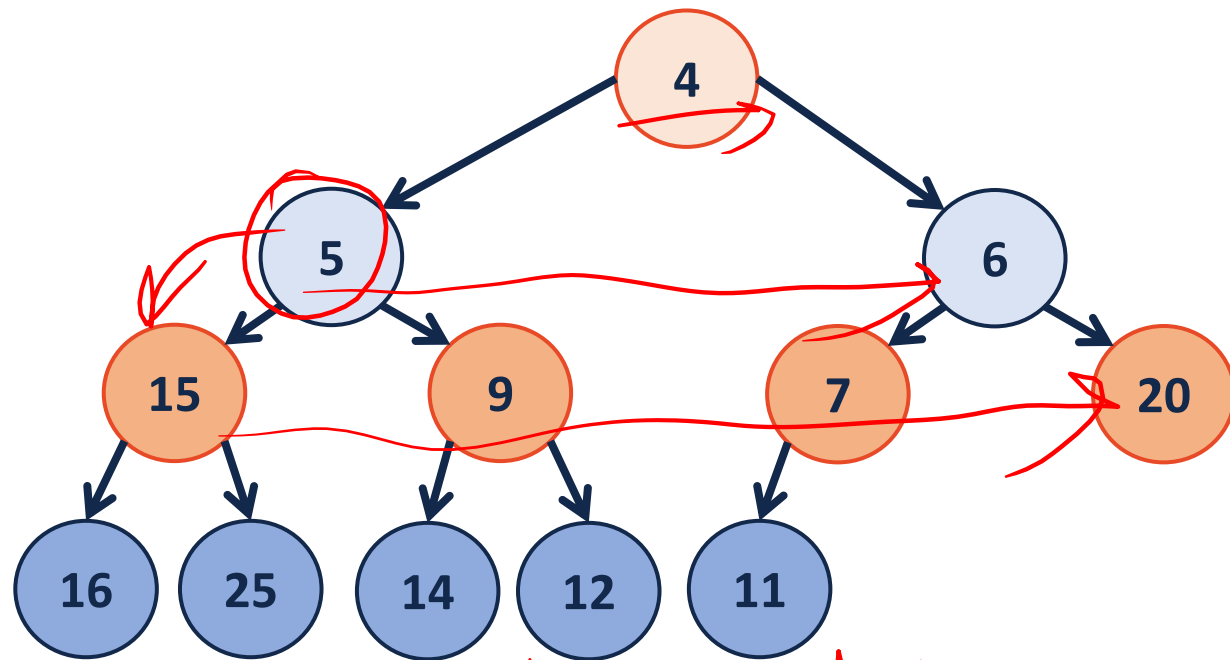
- $T = \{\}$ or
- $T = \{r, T_L, T_R\}$, where r is less than the roots of $\{T_L, T_R\}$ and $\{T_L, T_R\}$ are min-heaps.



visual model of
implementation

(min)Heap

Level order Trav
* In a heap impl
[0] is not used



Left child = $2 * \text{index}$
Right child = $2 * \text{index} + 1$

visual model
Parent = $\lfloor \text{index} / 2 \rfloor$

